INFORME DE EVIDENCIAS Y MONITOREO DE DESARROLLO (RAMA FRONTEND / INTEGRACIÓN)

Proyecto: ClearBid NJ – Sistema de Inteligencia Pre-Puja para Remates Inmobiliarios

Asignatura: Proyecto APT (Capstone)

Estudiante / Responsable Frontend: Cristopher Leonardo Hernández Calderón

Fecha: 20/10/2026

1. Resumen de Avance y Ajustes en el Frontend

Durante esta etapa, el trabajo del área de Frontend & UX se concentró en conectar la aplicación web construida en React 18, TypeScript, Tailwind CSS y Vite con la API REST de FastAPI y la base de datos PostgreSQL en Supabase.

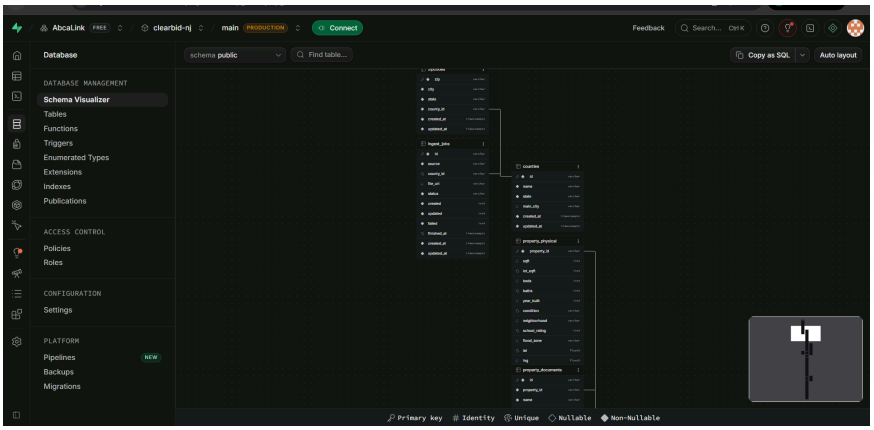
A diferencia de entregas anteriores donde solo existían bocetos estáticos, en este sprint se logró:

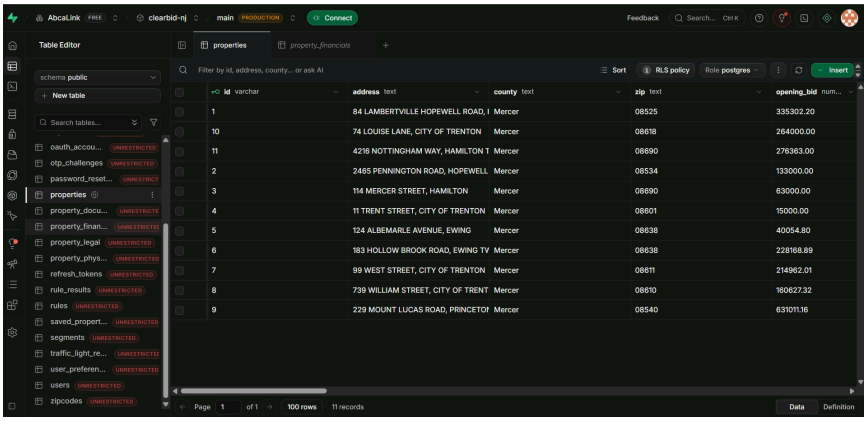
- **Conexión Real con Supabase:** La aplicación lee directamente el catálogo de propiedades registradas en la nube de Supabase a través del backend REST.
- **Visualización Dinámica de Fichas:** Despliegue de la tabla de subastas con precios de apertura (`Opening Bid`), ARV (Valor tras reparación), ROI estimado y estado del semáforo de riesgo.
- **Poblado con Caso de Uso Real:** Se cargaron en la base de datos los datos extraídos del reporte oficial de título de Mercer County (**4216 Nottingham Way, Hamilton Twp, Block 1853, Lot 19**)

2. Evidencias Visuales del Sistema Operativo

2.1. Arquitectura de Base de Datos y Persistencia en Supabase

Se materializó el esquema relacional en la plataforma de Supabase, incluyendo las tablas `properties`, `property_financials`, `property_legal`, `property_physical` y `traffic_light_results`.

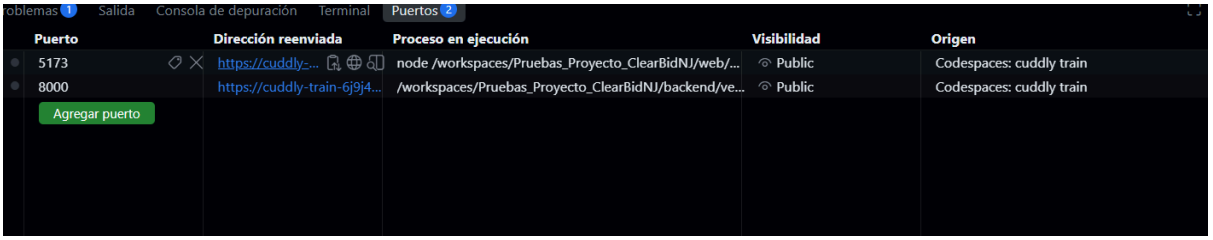


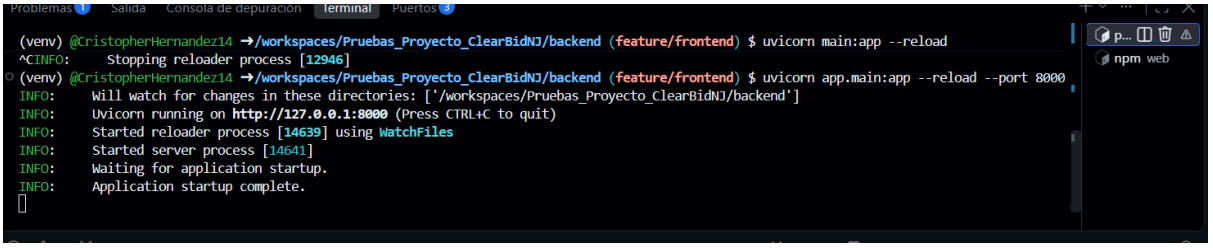


2.2. Servidores y Entorno de Desarrollo Local en GitHub Codespaces

Demostración de los servicios levantados localmente en la máquina virtual:

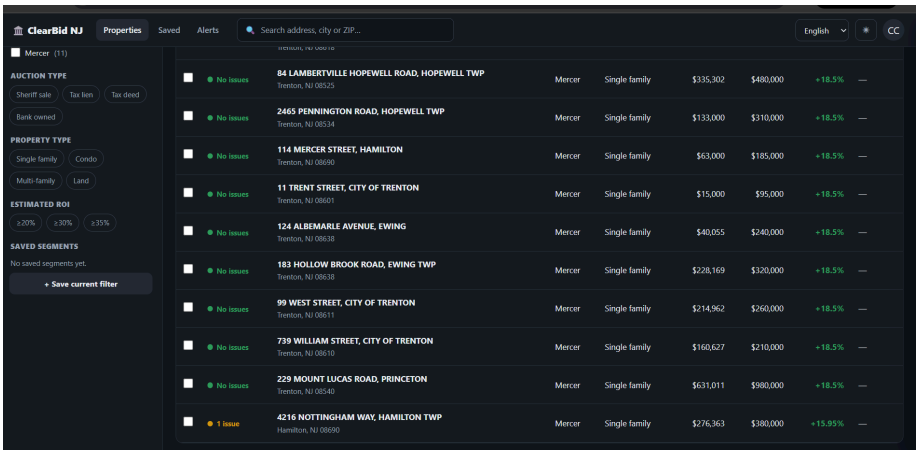
- **Backend FastAPI:** Corriendo en el puerto `8000` con `uvicorn app.main:app`.
- **Frontend React / Vite:** Corriendo en el puerto 5173 expuesto en modo público.





2.3. Dashboard e Interfaz Web de Propiedades (Frontend)

Despliegue final de la interfaz del usuario donde el inversor consulta el catálogo de subastas de Mercer County.



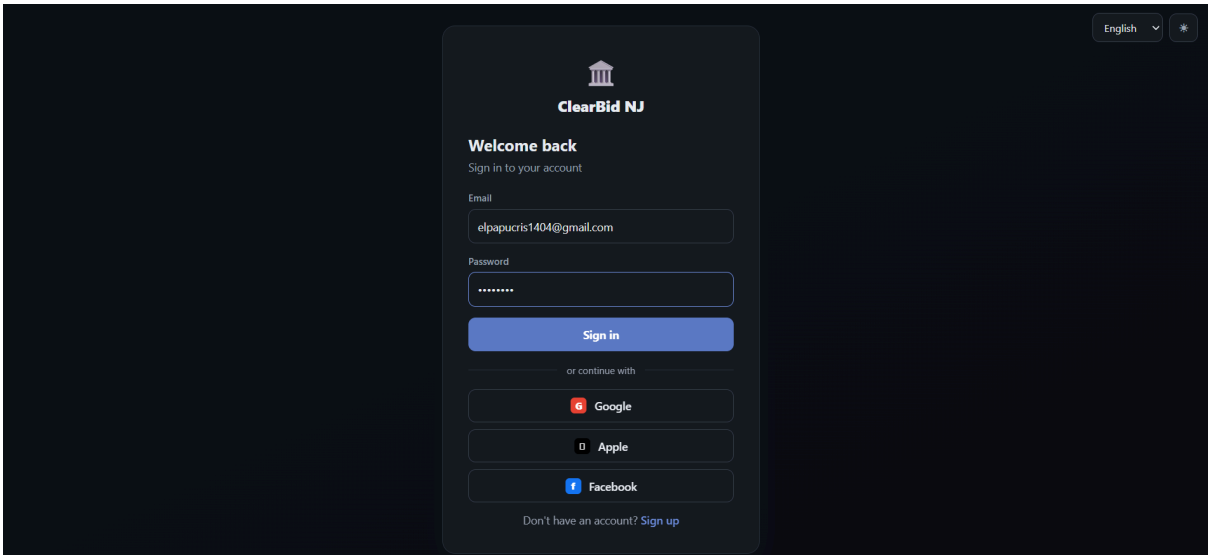
2.4. Seguridad, Autenticación JWT y Desafío 2FA

El sistema implementa un flujo de seguridad en dos pasos para proteger la plataforma:

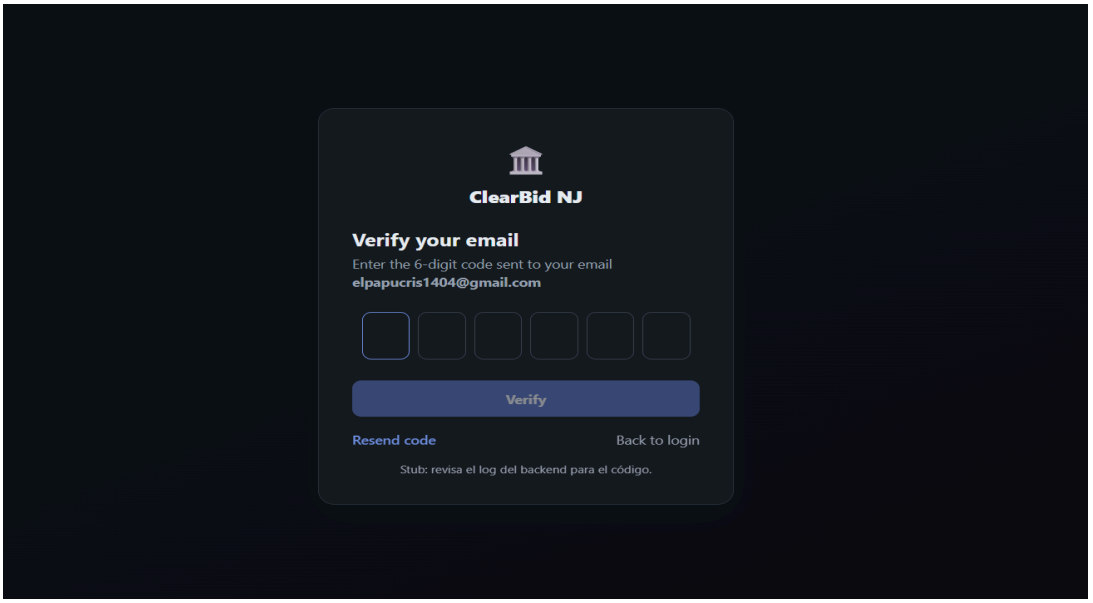
- **Login de Usuario:** Autenticación mediante credenciales con generación de tokens JWT rotativos y rutas protegidas en el Frontend con React Router y Zustand.
- **Verificación de Seguridad 2FA (OTP):** Generación y validación de código de verificación enviado al usuario/correo para autorizar la sesión activa antes de acceder al Dashboard de propiedades.

A continuación, quedará demostrado el funcionamiento de el login:

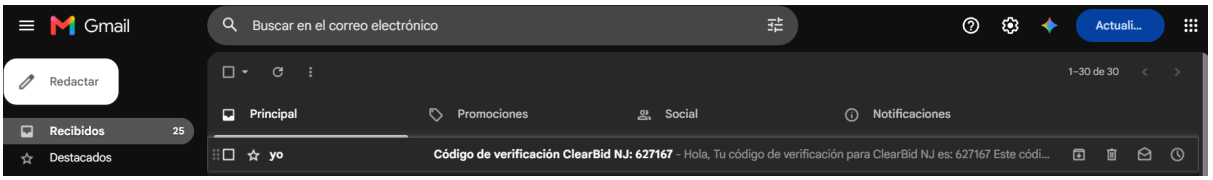
1. Se ingresan las credenciales correspondientes.



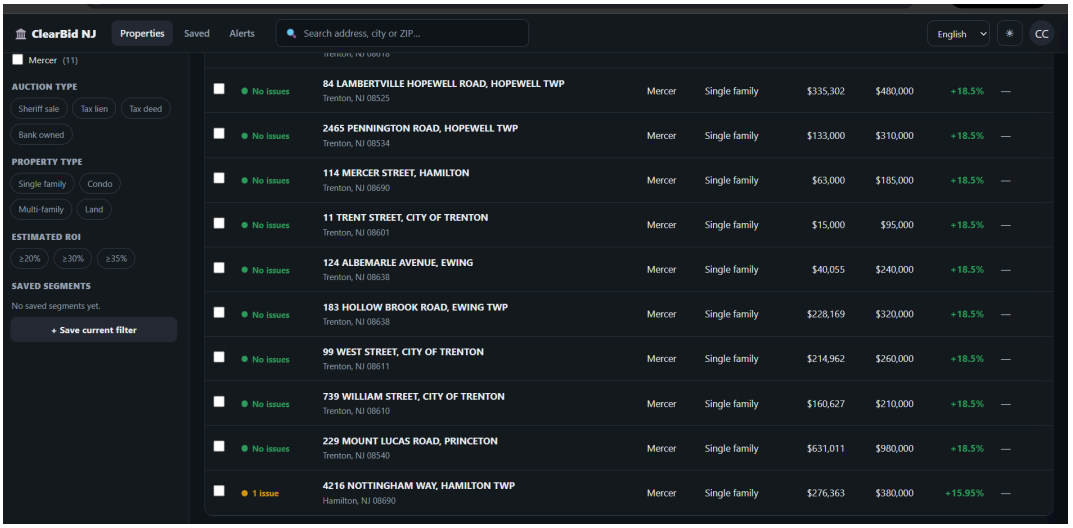
2. Al seleccionar el botón “Sign In” redireccionará al apartado de código, donde se espera que el código llegue al correo.



3. Se logra apreciar que el correo llega directamente a la bandeja de entrada, donde se permite ver el código de acceso.



4. Al poner el código en la interfaz de código , este redirecciona inmediatamente al dashboard principal donde se ven las propiedades.



3. Bitácora de Errores Detectados, Diagnóstico y Correcciones Realizadas.

Para demostrar rigor técnico y cumplimiento con los estándares de la industria, se documentan los principales desafíos resueltos durante la integración:

Error / Incidencia	Causa Técnica	Solución Implementada
HTTP Error 404 / Port Collision	Vite intentó arrancar en el puerto 5174 porque el 5173 quedó bloqueado por un proceso anterior.	Se liberaron los puertos en VS Code, configurando la exposición pública y la bandera `--host` para Codespaces.
HTTP 500 Internal Server Error (Pydantic ValidationError)	Discrepancia de tipos de datos en la API: Pydantic esperaba cadenas en minúsculas (`single_family`, `yellow`) y campos no nulos como `city` y `county_id`.	Se ejecutaron scripts de migración y normalización SQL en Supabase (`UPDATE properties SET city = 'Trenton', flag = 'yellow'`), alineando la BD con la validación de Pydantic.
Panel Lateral en Carga Indefinida (`Loading...`)	Ausencia de registros en las tablas hijas (`property_financials`, `property_physical`) para los IDs del 1 al 10 al hacer JOIN	Se poblaron las tablas secundarias en Supabase mediante cláusulas `INSERT INTO ... ON CONFLICT DO NOTHING`, permitiendo resolver los detalles de la propiedad.

3.1. Proceso de Depuración y Resolución De Conflictos de Integración.

Inicialmente, la integración entre el Frontend y Supabase presentó fallos de renderizado y respuestas de error `500 Internal Server Error` debido a tres factores principales que fueron corregidos:

- **Estado Anterior (Antes):** Inconsistencia en los esquemas de Pydantic por campos nulos (`city`, `county_id`) y valores en mayúsculas (`Single Family`, `YELLOW`). Esto provocaba que React solo mostrara una tarjeta o arrojara pantallas de error bloqueantes.
- **Acción de Mejora y Refactorización (Después):** Se aplicaron scripts de normalización en Supabase para alinear la base de datos con los enumerados requeridos (`single_family`, `yellow`), y se poblaron las tablas secundarias (`property_financials`, `property_physical` y `rule_results`).
- **Resultado:** Despliegue fluido de la lista completa de subastas (11 propiedades) con cálculo de ROI (+15.95%), semáforos activos y apertura del panel lateral de detalle sin estados de carga indefinidos.

4. Referencias y Fuentes Consultadas

- **Mercer County Sheriff's Office:** *Sheriff's Foreclosure Sales List and Guidelines*, New Jersey. Disponible en: https://www.mercercounty.org/government/sheriff-/informational/sheriff-s-foreclosure-sale.
- **Mercer County Clerk RecordsNG:** *Public Records Search for Title and Mortgages*. Disponible en: https://records.mercercounty.org/RecordsNG_Search/.
- **FastAPI Documentation:** *Pydantic Models and Data Validation*. Python Software Foundation.
- **React & Vite Guide:** *State Management with TanStack Query and Zustand in SPA Architecture*.